Optimization of a City Transportation Network using MST

Student: Nurdan Zhanabayev

Introduction

The goal is to optimize a city's transportation network by connecting all districts with the lowest possible construction cost.

This is achieved using **Minimum Spanning Tree (MST)** algorithms — **Prim's** and **Kruskal's** — applied to weighted undirected graphs where:

- Vertices represent city districts.
- Edges represent potential roads.
- Weights represent construction costs.

1. Summary of Input Data and Algorithm Results

Graph ID	Vertices	Edges	Connected	Algorithm	Total Cost	Operations	Execution Time (ms)
1	4	5	true	Prim	7.00	7	2.784
1	4	5	true	Kruskal	7.00	28	0.389
2	5	7	true	Prim	16.00	10	0.061
2	5	7	true	Kruskal	16.00	40	0.040
3	7	8	true	Prim	17.00	13	0.068
3	7	8	true	Kruskal	17.00	50	0.042

4	8	9	true	Prim	20.00	15	0.062
4	8	9	true	Kruskal	20.00	57	0.047

Observations:

1. Connectivity:

All test graphs were fully connected (true), allowing both algorithms to successfully compute a Minimum Spanning Tree (MST) without encountering disconnected components. This verifies the correctness of the input datasets and the robustness of the implementations for connected networks.

2. Correctness of Results:

For every dataset, both Prim's and Kruskal's algorithms produced identical total MST costs, confirming algorithmic correctness and logical consistency. This demonstrates that both methods lead to the same minimum-cost configuration, even though they build the MST differently (Prim grows one tree; Kruskal merges multiple components).

3. Execution Time Comparison:

Kruskal's algorithm consistently showed lower execution times (measured in milliseconds), especially on smaller graphs.

   a. The faster runtime is due to efficient edge sorting and union-find operations, which work well when the number of edges is limited.
   b. Prim's algorithm performed slightly slower because each iteration involves scanning or heap operations for adjacent vertices.
   c. However, the time gap between the two algorithms remains minor for small datasets (<10 vertices).

4. Operation Count and Efficiency:

Prim's algorithm performed fewer total operations across all graph sizes.

   a. The difference between the algorithms grew as the number of vertices increased: Kruskal required roughly 3–4× more operations, consistent with its need to sort all edges and perform frequent union-find checks.
   b. This confirms that Prim is computationally leaner per operation, while Kruskal executes many more lightweight operations.

5. Scalability Trends:
    a. As graph size and edge count increased, Prim's operation count rose gradually, while Kruskal's increased more steeply due to edge sorting overhead.
    b. If tested on larger graphs (20–30+ vertices), this pattern would likely result in Prim becoming more time-efficient for dense networks.
6. Performance Trade-Off:
    a. Kruskal is faster on small, sparse graphs where edge sorting dominates the workload.
    b. Prim is more stable and operation-efficient as graphs become denser or larger.
    c. Both algorithms remain reliable, with performance differences primarily driven by graph density and data representation.

2. Practical Comparison of Efficiency and Performance

Criterias	Prim's Algorithm	Kruskal's Algorithm
Time complexity	$O(V^2)$ (simple) or O(E log V) (with PQ)	O(E log E) or O(E log V)
Observed execution time (ms)	Slightly higher (0.06–0.18 ms)	Slightly lower (0.04–0.10 ms)
Operations count	~3–4× fewer operations	~4× more operations (due to sorting edges)
Graph density handling	Better for dense graphs (many edges)	Better for sparse graphs (fewer edges)
Implementation	Simple with adjacency matrix or list	sorting and disjoint-set union (DSU)

Memory usage	Lower (stores adjacency structure)	Higher (stores full edge list and DSU)
Practical results	More efficient in operations, less code complexity	Faster runtime on small and medium sparse graphs

Metric	Derived From	What It Shows
Ops-to-Time Ratio	OperationsCount / ExecutionTimeMs	How many operations executed per ms — higher means tighter loops or faster operations.
Scaling Behavior	Compare Ops and Time as Vertices ↑	Shows how performance grows with graph size ($O(V^2)$ or O(E log V) patterns).

Prim (adjacency matrix)

$T(V) = \sum_{i=1}^{V} O(V) = O(V^2)$

- The algorithm selects the smallest edge that connects a new vertex to the growing tree.
- With an adjacency matrix, each time we add a new vertex, we must scan all V vertices to find the smallest connection.
- This happens V times (once per vertex).
- So total time = V × O(V) = $O(V^2)$.
- Efficient for dense graphs because most edges exist anyway.

Prim (adjacency list + min-heap)

Extract-min: V × O(log V) = O(V log V)

Heap updates (insert/decrease-key): ≤ O(E) × O(log V) = O(E log V)

T = O(V log V + E log V) = O((V+E) log V) = O(E log V)

- Instead of scanning all vertices, we use a *priority queue (min-heap)* to always extract the smallest edge faster.
- Extracting the smallest element from the heap takes O(log V), done V times → O(V log V).
- Updating or inserting edges in the heap happens for each edge (E times), each costing O(log V) → O(E log V).
- Total time: O(V log V + E log V) = O((V+E) log V).
- For connected graphs, E ≥ V, so simplified to O(E log V).
- This version is faster for large sparse graphs.

Kruskal

Sort edges: O(E log E)

Union-Find (find/union) per edge: O(E α(V)) = O(E) (α = inverse Ackermann)

T = O(E log E + E) = O(E log E) = O(E log V) (since log E = Θ(log V))

- Works by sorting all edges by weight, then adding them one by one if they don't form a cycle.
- Sorting edges costs O(E log E).
- For each edge, we check and merge components using *Union-Find* structure; each operation is almost constant time (α(V)).
- Total: O(E log E + E) ≈ O(E log E).
- Since log E ≈ log V for most graphs, we can write O(E log V).
- Performs better on sparse graphs because it only looks at edges.

Final forms:

- Prim (matrix): $O(V^2)$ → best for dense graphs.
- Prim (heap): O(E log V) → efficient for large graphs.
- Kruskal: O(E log E) ≈ O(E log V) → efficient for sparse graphs.

Example (Graph 10):

- Prim → 47 ops, 0.186 ms
- Kruskal → 171 ops, 0.108 ms

Even though Kruskal is faster, it does ~3.6× more work, confirming that **Prim's operations are heavier but fewer**, while **Kruskal's are lighter but many**.

  - Lower operationsCount → algorithm did less computational work.
  - In your CSV, Prim uses **7–47 operations**, while Kruskal uses **28–171**, meaning Kruskal performed **3–4× more internal steps**.
  - Even if Kruskal is faster in milliseconds, Prim is more efficient per operation.

In practice:

Even though Kruskal's algorithm executed slightly faster in milliseconds, this difference is marginal and primarily influenced by the low graph sizes. Prim's algorithm performed fewer internal operations, meaning it scales better when many edges exist (dense graphs).

For sparse graphs ($E \approx V$), Kruskal's simplicity and edge-based nature make it slightly faster. For dense graphs, Prim's priority queue approach becomes preferable.

3. Conclusions

Correctness:

Both Prim's and Kruskal's algorithms consistently produced identical MST total costs for all tested datasets, confirming correctness of implementation and equivalence in final output.

Efficiency and Performance:

- **Kruskal's algorithm** was faster in raw execution time on small and medium **sparse graphs**, where the number of edges (E) is close to the number of vertices (V).
  - Sorting edges once ($O(E \log E)$) is inexpensive when E is small.
  - The Union-Find structure efficiently merges components with almost constant-time operations, resulting in good performance.
- **Prim's algorithm** performed fewer total operations and scales better with increasing graph density.

- In dense graphs ($E \approx V^2$), Prim avoids global edge sorting and only processes edges incident to the current tree, which reduces overhead.

Graph Density:

- For **sparse graphs (E ≈ V)** → **Kruskal** is preferable. It processes fewer edges and benefits from efficient edge sorting.
- For **dense graphs ($E \approx V^2$)** → **Prim** is preferable, especially when implemented with an adjacency list and min-heap. It avoids O(E log E) sorting cost and performs near $O(V^2)$ or O(E log V) time.

Edge Representation and Data Structures:

- **Prim** integrates naturally with adjacency matrices or adjacency lists and uses a **priority queue (min-heap)** to select the next minimum edge efficiently.
- **Kruskal** depends on a **global edge list** and the **Disjoint-Set Union (DSU)** structure to detect cycles. This makes it slightly more complex to implement but flexible when the graph is already stored as an edge list.

Implementation Complexity and Memory:

- **Prim's algorithm** is simpler to implement when a graph is represented as an adjacency list or matrix, requiring less additional data structure overhead.
- **Kruskal's algorithm** needs extra memory for the DSU structure and the sorted edge array, which can be costly for very dense graphs.

Scalability and Practical Use:

- In large-scale transportation or network systems where graphs are dense and fully connected, **Prim with a min-heap** provides better asymptotic scaling and predictable performance.
- In datasets where connections are sparse (e.g., rural networks, pipelines, communication towers), **Kruskal** performs faster because fewer edges are processed and union-find operations are efficient.
- For extremely large graphs stored as edge lists (common in real datasets), Kruskal is often easier to parallelize and apply incrementally.

Overall Conclusion:

- **Kruskal** is ideal for small or sparse graphs and when the input is naturally given as an edge list.
- **Prim (with heap)** is ideal for dense or matrix-based graphs and large connected networks.
- Both algorithms are correct and efficient; the preferable choice depends on graph structure, data representation, and required scalability.

4. References

1. FreeCodeCamp. (2021). *Prim's Algorithm explained with pseudocode*. https://www.freecodecamp.org/news/prims-algorithm-explained-with-pseudocode/
2. GeeksforGeeks. (n.d.). *Prim's vs Kruskal's Algorithm – Comparison and Implementation*. https://www.geeksforgeeks.org/
3. OpenStax. (2020). *Data Structures and Algorithms*. OpenStax CNX. https://openstax.org/books/introduction-computer-science/pages/3-1-introduction-to-data-structures-and-algorithms?query=Prim
4. Programiz. (n.d.). *Kruskal's Algorithm*. https://www.programiz.com/dsa/kruskal-algorithm
5. Programiz. (n.d.). *Prim's Algorithm*. https://www.programiz.com/dsa/prim-algorithm